Machine Learning for Fault Detection on the Tennessee Eastman Process

An Engineer's Case Study

Mrinal Gala

Avisha.AI — Intelligence, Thoughtfully Applied

Abstract

Early detection of process faults is important for the safe and efficient operation of chemical plants, particularly when faults develop gradually and are difficult to distinguish from normal process variation. This study applies machine learning to fault detection on the Tennessee Eastman Process, a widely used benchmark for process monitoring. A Random Forest classifier was used to distinguish normal from faulty operation, and two feature sets were compared: static features based on instantaneous sensor measurements and time-aware features incorporating short-term trends and changes in variability. All results were obtained using a leakage-safe evaluation protocol in which entire simulation runs were assigned wholly to either the training or test set. At first, the time-aware model looked remarkably good. Once the evaluation was tightened, the picture became more realistic. Adding temporal context produced only a small increase in overall accuracy but substantially reduced missed faults, albeit at the cost of additional false alarms. The largest improvements were observed for the most difficult faults, although these remained the hardest to detect. The findings show that, in process fault detection, the types of errors a model makes are often more informative than overall accuracy alone. More broadly, the study highlights the importance of careful feature design, rigorous evaluation, and engineering interpretation when developing fault detection systems for practical use.

1. Introduction

A chemical plant never stops. It runs around the clock, monitored by hundreds of sensors reading temperature, pressure, flow, and level across reactors, separators, strippers, and the recycle loops that tie them together. That coupling is the heart of the problem: when something goes wrong — a feed cuts out, a valve sticks, a sensor begins to drift — the disturbance rarely stays where it started. A fault that begins in one unit can trigger controller responses, alter conditions elsewhere in the plant, and gradually develop into a problem whose symptoms appear far from the original cause.

Not all faults are equal. Some are loud: a variable jumps clean out of its normal band, and any threshold will catch it. The dangerous ones are quiet — a slow drift or a small change in variability that hides inside the ordinary noise of a running plant and never trips a single-variable limit. An operator watching one gauge will miss them. This is where a model that observes every sensor at once, and watches how they move together over time, earns its place.

This study tests that idea on the Tennessee Eastman Process. The approach is data-driven and adds no instrumentation; it learns from the measurements the plant already produces. Three questions are asked: which faults are easy to detect and which are hard, whether short-term temporal context improves detection of the hard ones, and how much of any apparent improvement survives a genuinely honest evaluation. The third question proves to matter as much as the first two.

2. Related Work

The Tennessee Eastman Process is one of the most heavily studied benchmarks in process fault detection, and the methods applied to it fall into three broad generations. The first is multivariate statistical monitoring — principal component analysis, partial least squares, and related projections — applied systematically to the process by Chiang, Russell, and Braatz (2001) and set within the wider fault detection and diagnosis literature by Venkatasubramanian et al. (2003). The second is classical machine learning, such as the support vector machine approach of Kulkarni et al. (2005). The third, and most recent, is deep learning: convolutional and recurrent architectures that learn temporal structure directly from data (for example, Wu and Zhao, 2018). Many of these methods report high detection rates on the benchmark.

Two things tend to be lost in the pursuit of higher rates. The first is evaluation discipline. Data leakage — allowing information from the test set to influence training — is a well-documented cause of over-optimistic results in machine learning (Kaufman et al., 2012), and it is easy to introduce when samples from the same simulation run are split across the training and test sets: the model learns to recognise the run rather than the fault. The second is physical interpretation: a detection rate reveals nothing about why a fault is hard to catch, or what an operator should do once it is flagged. This study is positioned deliberately in that gap. It does not aim to surpass the benchmark's best reported scores. It asks instead what an honestly evaluated, physically interpreted detector looks like — and where it still falls short.

3. Data

The Tennessee Eastman Process simulates a continuous plant of five coupled units — a reactor, condenser, vapour–liquid separator, stripper, and recycle compressor — that convert gaseous reactants into two liquid products. It was introduced as a control and monitoring challenge problem (Downs and Vogel, 1993), and it has endured as a benchmark because it captures what makes real plants difficult: nonlinearity, recycle, and closed-loop control that actively responds to disturbances. Figure 1 shows the layout.

This study uses the public simulation dataset of Rieth et al. (2017). Each sample records 52 variables — 41 process measurements (temperatures, pressures, flows, levels, and compositions) and 11 manipulated variables (valve and actuator settings) — recorded every three minutes. The dataset covers normal operation and twenty fault scenarios spanning feed-composition changes, equipment malfunctions, sensor biases, and valve faults. Fault 6, a step loss of the A reactant feed, is used as a worked example throughout this report. In each faulty run the simulation begins under normal operation, with the fault introduced after the twentieth sample; samples are labelled accordingly, early ones as normal and later ones as faulty.

One deliberate choice shaped the work that followed: the dataset's coded variable names (e.g., xmeas_16) were mapped to their physical meanings (e.g., Stripper Pressure) using the standard Tennessee Eastman Process variable definitions, so that model behaviour could be reasoned about as process engineering rather than as anonymous columns of data. The value of this becomes clear with Fault 6. In practice, a step loss of the A reactant feed does not simply reduce one flow measurement. It changes reactor conditions, affects downstream separation, and prompts control actions throughout the process — a single local disturbance influencing the wider plant, and a pattern that appears across many of the correlated variables the model learns from.

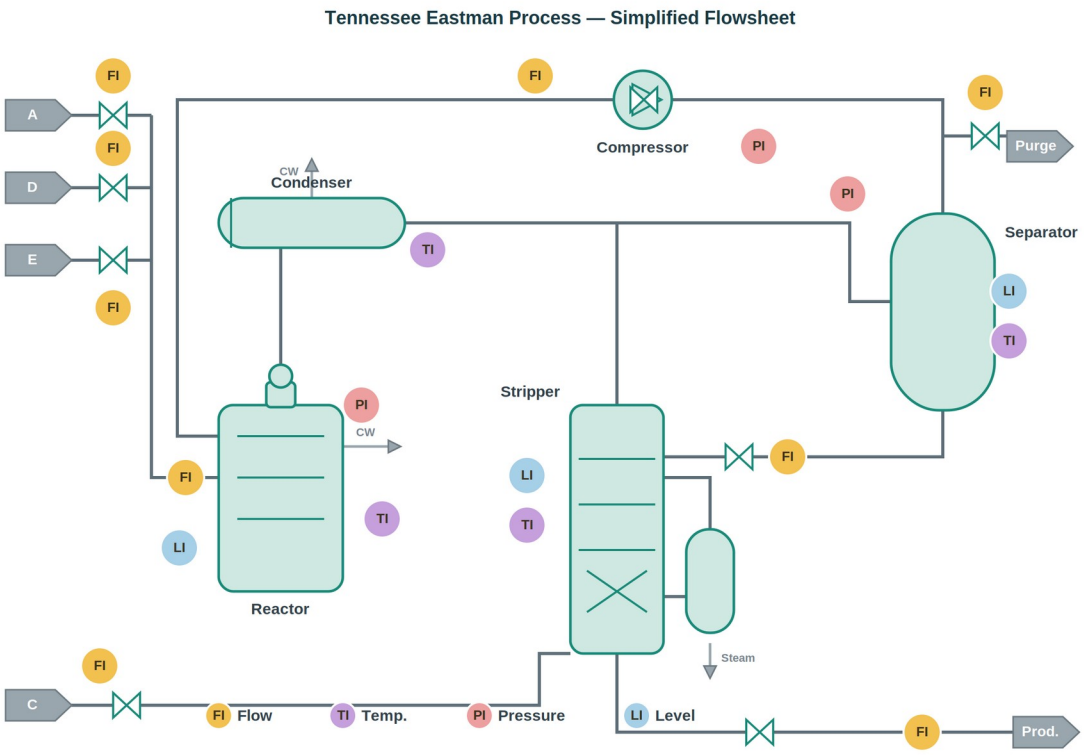


Figure 1. Simplified flowsheet of the Tennessee Eastman Process, showing the five major units and representative instrumentation (flow, temperature, pressure, and level indicators).

4. Method

Detection is framed as binary classification: at each time step, the model decides whether the plant is operating normally or under a fault. Fault-free samples are labelled normal (0). Within a faulty run, samples after the fault onset (sample > 20) are labelled faulty (1), while the pre-onset samples are retained as additional normal examples — reflecting the physical reality that, for its first few minutes, a faulty run is genuinely indistinguishable from a healthy one.

Two feature sets are compared. Static features use only the 52 instantaneous variables — the raw process snapshot at a single time step. Time-aware features add short-term memory: for each variable, a rolling mean, a rolling standard deviation, and a rate-of-change term computed over a window of 15 samples. These rolling features are calculated strictly within each simulation run, so that no information crosses between independent runs.

The classifier is a Random Forest of 300 trees, chosen deliberately. It handles nonlinear relationships, tolerates correlated and noisy variables, requires little tuning, and — unlike a deep network — can be interrogated about which variables drove a given decision. For a study whose central concern is interpretability, that last property is not a minor convenience.

Evaluation follows one rule, applied strictly: no run appears on both sides of the split. Rather than dividing individual samples — which would allow a run's rolling features to appear in both training and test — entire simulation runs are assigned wholly to one side, roughly 80% to training and 20% to test. Three quantities are reported: overall accuracy; the per-fault detection rate, defined as the fraction of a fault's faulty samples correctly flagged; and the division of errors into false

alarms (normal operation flagged as faulty) and missed faults (faulty operation flagged as normal). In a real plant these two errors are not equal in consequence, and much of what follows turns on the difference.

5. Results

On identical, unseen test runs, static features alone reached an overall accuracy of 0.849, with 358 false alarms and 871 missed faults. Time-aware features raised accuracy only to 0.855 — a number that, taken alone, would suggest the temporal features barely helped. The composition of the errors tells a different story. Missed faults fell from 871 to 492, while false alarms rose from 358 to 690. The time-aware model trades a larger number of tolerable false alarms for far fewer dangerous missed faults. In a plant, where a missed developing fault can lead to off-spec production, equipment damage, or a safety incident, this is the trade worth making.

Accuracy, then, is a misleading headline. Two models of nearly identical accuracy can fail in entirely different ways, and it is the missed-fault rate — not overall accuracy — that most directly reflects operational risk.

A clear pattern emerges from Table 1. Faults that were already easy to detect from an instantaneous snapshot — Faults 1, 2, 4, 6, and 7 — show little or no benefit from temporal features. The largest gains occur among the most challenging faults, particularly Faults 3, 9, and 15, whose detection rates increase roughly three- to four-fold. Even so, these three remain the hardest in the set, with roughly half of their faulty samples still undetected — a limitation revisited in the Discussion. The pattern suggests that temporal context is most valuable when fault signatures are weak and develop gradually, allowing the classifier to exploit trends and changes in variability that a single process snapshot cannot reveal.

Fault	Static detection rate	Time-aware detection rate
1	1.00	1.00
2	1.00	1.00
3	0.14	0.51
4	1.00	1.00
5	0.78	0.92
6	1.00	1.00
7	1.00	1.00
8	0.97	0.98
9	0.15	0.50
10	0.86	0.95
11	0.88	0.92
12	0.98	0.98
13	0.90	0.93
14	0.99	1.00

Fault	Static detection rate	Time-aware detection rate
15	0.11	0.48
16	0.79	0.93
17	0.96	0.97
18	0.89	0.91
19	0.70	0.82
20	0.78	0.88

Table 1. Per-fault detection rate, static vs. time-aware features (leakage-safe split).

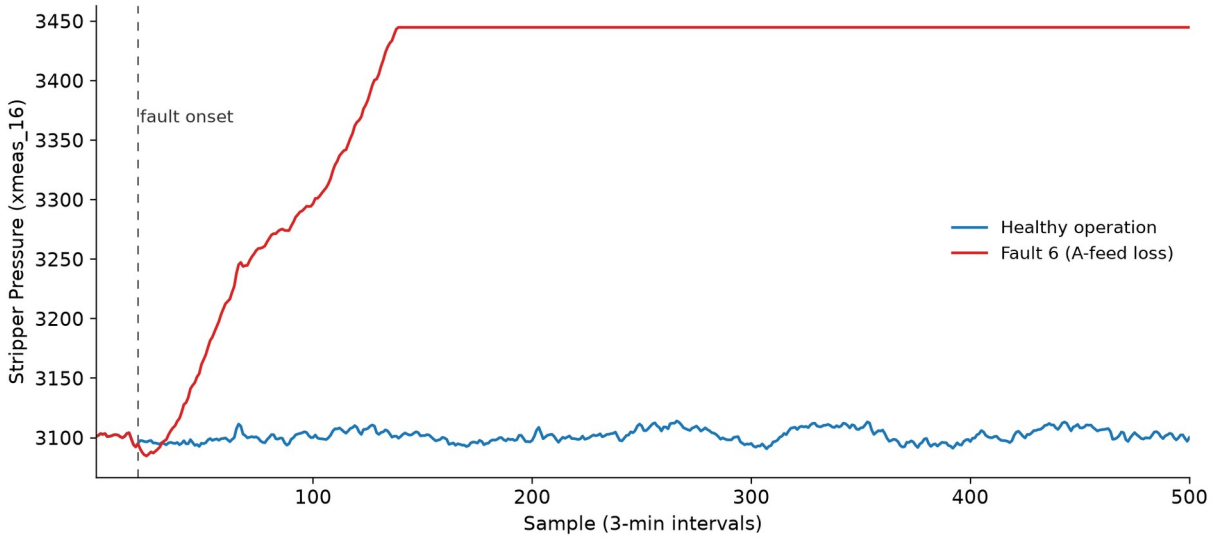


Figure 2. Stripper pressure (xmeas_16) under healthy operation and under Fault 6 (a step loss of the A reactant feed). The faulty trace departs from normal operation shortly after fault onset (sample 20) and rises to a new steady level.

6. Discussion

The two models score almost the same overall, yet the time-aware model is the better one, and the reason is physical rather than statistical. A developing fault is a trajectory, not a state. At any single instant, a slowly drifting process can appear normal — every gauge within its band — while the system is already sliding toward failure. This is why an instantaneous snapshot is structurally blind to a whole class of faults, and why trends and changes in variability are the only place their signal can be found.

Faults 3, 9, and 15 are where this argument meets its limit. Their signatures barely rise above the normal noise of the process; the change they introduce in any single variable is small, gradual, and easily mistaken for ordinary fluctuation. Temporal features help — a shallow trajectory still differs from noise — but where the trajectory is this faint, even trends separate faulty from normal operation only partially. These are precisely the faults a human operator scanning individual readings would be most likely to miss, which is the strongest case for an automated detector, and at the same time an honest measure of how much difficulty remains.

None of this is free. The same shift that reduced missed faults nearly doubled the false alarms, and false alarms carry their own cost. An operator flooded with nuisance alarms begins to distrust the system, and a detector that is routinely ignored protects nothing — the well-known problem of alarm fatigue. The trade therefore favours safety only while false alarms remain tolerable; beyond a certain threshold, they erode the very trust the system depends on. Where that operating point is set is an engineering decision, made against the specific consequences and tolerances of the plant, and ideally with the operators who will live with the alarms.

One methodological point deserves emphasis, because it changed the study's own conclusions. When the time-aware features were first evaluated using a sample-level split, the hardest faults appeared to reach detection rates near 0.80. That improvement was an illusion. Because the rolling features draw on neighbouring samples, splitting at the level of individual samples allowed information from the same run to appear in both the training and test sets — the model was effectively reading part of its own answer key. Re-evaluating with a run-level split removed the leakage and returned the honest figures reported here, with the hardest faults closer to 0.50. The inflation affected the time-aware features far more than the static ones, precisely where the rolling window could borrow from adjacent samples. The lesson extends beyond this dataset: whenever features carry temporal context, the unit of splitting must be the whole run rather than the individual sample, or reported performance can be substantially overstated.

This study has clear limits. It runs on a simulation — faithful to plant dynamics, but a simulation nonetheless, and strong results here do not guarantee the same on physical hardware. It detects faults but does not isolate them: it reports that something is wrong, not which fault has occurred. And it treats faults one at a time, cleanly separated. Real plants are rarely so tidy. Faults interact, control systems respond, operators intervene, and what begins as a single disturbance can evolve into a sequence of events far more complex than the initial fault alone. Capturing that behaviour remains one of the most important challenges for future work.

7. Conclusion & Future Work

This study applied machine learning to fault detection on the Tennessee Eastman Process under a strict, leakage-safe evaluation, and its central finding concerns the shape of the errors more than their count. Temporal context barely moves overall accuracy, but it reshapes the model's failures toward safety — cutting dangerous missed faults and lifting the hardest faults from nearly undetectable to roughly half-caught. The honest evaluation is the backbone of that claim: it is what separates a result that can be defended from one that disappears on real data. This is a case study rather than a new method — the benchmark is well established and the tools are standard. Its value lies in carrying the full chain through carefully: feature design, leakage-safe evaluation, and engineering interpretation, joined so that the findings are both technically sound and operationally meaningful.

Several directions extend naturally from this work. The most immediate is the move from detection to isolation — from reporting that something is wrong to identifying which fault has occurred — turning the binary detector into a diagnostic one. In industrial settings, operators must often distinguish a primary fault from the secondary responses it triggers, which makes isolation substantially harder than detection alone. A further step is a prescriptive layer that maps a diagnosed fault to a recommended corrective action, where process-engineering knowledge contributes most directly: the appropriate response to a loss of reactant feed follows from

understanding the process, not the data alone. On the modelling side, sequential architectures such as temporal convolutional networks or LSTMs, and ultimately physics-informed hybrids that combine learned models with mass and energy balances, offer a route to detectors that encode process structure rather than infer it entirely from data. Finally, applying the approach to a different simulation or, ideally, to real plant data would test how far these findings generalise — the point at which a demonstration of capability becomes a contribution of independent value.

References

Chiang, L. H., Russell, E. L., & Braatz, R. D. (2001). Fault Detection and Diagnosis in Industrial Systems. Springer-Verlag, London.

Downs, J. J., & Vogel, E. F. (1993). A plant-wide industrial process control problem. Computers & Chemical Engineering, 17(3), 245–255.

Kaufman, S., Rosset, S., Perlich, C., & Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. ACM Transactions on Knowledge Discovery from Data, 6(4), 1–21.

Kulkarni, A., Jayaraman, V. K., & Kulkarni, B. D. (2005). Knowledge incorporated support vector machines to detect faults in Tennessee Eastman Process. Computers & Chemical Engineering, 29(10), 2128–2133.

Rieth, C. A., Amsel, B. D., Tran, R., & Cook, M. B. (2017). Additional Tennessee Eastman Process Simulation Data for Anomaly Detection Evaluation. Harvard Dataverse. https://doi.org/10.7910/DVN/6C3JR1

Venkatasubramanian, V., Rengaswamy, R., Kavuri, S. N., & Yin, K. (2003). A review of process fault detection and diagnosis: Part III: Process history based methods. Computers & Chemical Engineering, 27(3), 327–346.

Wu, H., & Zhao, J. (2018). Deep convolutional neural network model based chemical process fault diagnosis. Computers & Chemical Engineering, 115, 185–197.